Member 3 Briefing: FP32 and CUDA Variants

***Beginner foundation: start here before reading about GPU files or number formats.** The point is to understand why the project has variants at all.*

Beginner foundation: one task, different machines and number formats

The project always tries to solve the same mathematical problem, A*x=b. However, it can run that work on different kinds of processor and use different ways to store decimal-like numbers.

CPU and GPU

A CPU is the computer's general-purpose processor. It is good at many different kinds of tasks. A GPU has many smaller processing units designed to perform lots of similar calculations at the same time. Repeated operations such as sparse matrix-vector multiplication can be suitable for a GPU.

FP64 and FP32

Computers do not store arbitrary real numbers perfectly. They use floating-point formats. FP64 (double) keeps roughly 16 decimal digits; FP32 (float) keeps roughly 7. FP32 can use less memory, but it rounds more aggressively and cannot safely represent every value that FP64 can represent.

CUDA and .cu files

CUDA is NVIDIA's system for writing programs that use NVIDIA GPUs. A .cu file is CUDA C++ source code. It may include GPU calculations and CPU-side code that prepares those calculations. CUDA is not a new solver algorithm; it is a way to execute parts of the solver on a GPU.

Why variants are separate

Changing FP64 to FP32 can change rounding, overflow behavior, and what error level is realistic. Moving work to a GPU can change the order of arithmetic. That is why the project has separate implementations and separate tests rather than assuming every version behaves identically.

The remaining sections explain the exact files, tolerances, limitations, and safe presentation wording.

GPU words decoded before they appear

Word - Beginner meaning

Driver - Software that lets the operating system communicate with a GPU.

CUDA toolkit - NVIDIA's compiler, libraries, and tools for CUDA programs.

Kernel - A function designed to run many times in parallel on the GPU.

Host - The CPU side that starts and coordinates GPU work.

Device - The GPU side that performs GPU calculations.

Memory transfer - Moving data between CPU and GPU memory.

Backend - The implementation route: CPU or GPU.

Bit-for-bit equal - Every stored binary digit matches exactly. This is stronger than numerical agreement.

One simple rule

FP32 and CUDA are implementation choices, not new mathematical algorithms. They should preserve the intended method while accepting that precision and hardware can change rounding and performance behavior.

Your role in one sentence

Explain the separate FP32 implementation and CUDA backend without overstating equivalence, performance, or research novelty.

The project variants

The build produces four main solver programs:

```
cpu_fp64_ir  CPU, double precision
cpu_fp32_ir  CPU, float precision
gpu_fp64_ir  CUDA GPU, double precision
gpu_fp32_ir  CUDA GPU, float precision
```

All use the same high-level pipeline:

```
Matrix input -> SPAI preconditioner -> left-preconditioned GMRES
             -> iterative refinement -> original-system diagnostics
```

However, they are not identical in arithmetic behavior or implementation details.

1. Why FP32 is not a double to float text replacement

FP64 has about 16 decimal digits of precision; FP32 has about 7. Their unit roundoffs are:

$$u_{64} = 2^{-53}, \qquad u_{32} = 2^{-24}.$$

That changes input range, rounding, QR behavior, SPAI candidate scores, GMRES orthogonalization, and attainable residuals. Therefore FP32 has a separate API and implementation:

FP32 CPU file - Responsibility

include/bspai/fp32.hpp - FP32 CSR matrix, options, result records, and interfaces.

src/fp32_cpu.cpp - FP32 CSR, SPAI, GMRES, and iterative refinement.

The FP32 command-line path first reads canonical Matrix Market entries through the shared FP64 parser, then explicitly converts each value to float. It rejects entries that overflow to infinity or underflow from a nonzero input to zero. This prevents silent alteration of the matrix pattern.

Its main tolerances are different:

Quantity - FP64 - FP32

GMRES relative tolerance - 1e-8 - 1e-4

Outer backward-error goal - 10*2^{-53} - 10*2^{-24}

FP32 still follows the same broad SPAI-GMRES-IR algorithm, but the code uses float values, float diagnostics, and precision-appropriate checks.

2. CUDA source layout

The CUDA variants separate work by numerical subsystem:

File - FP64 responsibility

src/cuda_spmv.cu - GPU sparse matrix-vector multiplication.

src/cuda_spai.cu - GPU SPAI construction: scaling, QR-related work, pattern construction, assembly.

src/cuda_gmres.cu - GPU GMRES operations.

src/cuda_ir.cpp - GPU FP64 solver integration; outer refinement is CPU-staged.

include/bspai/cuda_*.hpp - CUDA-facing interfaces.

The matching FP32 CUDA files are:

File - Responsibility

src/fp32_cuda_spmv.cu - FP32 GPU SpMV.

src/fp32_cuda_spai.cu - FP32 GPU SPAI specialization.

src/fp32_cuda_gmres.cu - FP32 GPU GMRES.

include/bspai/fp32_cuda_*.hpp - FP32 CUDA interfaces.

The FP32 GPU SPAI implementation shares a constructor body through compile-time type/API mapping with the FP64 version. This reduces duplicated structure, but both precision paths still need separate tests.

3. What is on the GPU and what remains on the CPU?

The GPU backend accelerates numerical kernels such as sparse matrix-vector products, SPAI construction steps, and GMRES work. The project guide states that the GPU outer iterative-refinement loop is CPU-staged. Therefore, do not describe the implementation as a fully GPU-resident iterative-refinement loop.

The CPU and GPU can legitimately choose slightly different sparse patterns at exact score ties or report different singular-QR statuses. Floating-point operation order differs across platforms; bit-for-bit identity is not the correct expectation. The correct expectation is agreement with defined numerical acceptance checks.

4. Build configuration

The root CMakeLists.txt builds CPU targets. CUDA is optional through:

```
-DBSPAI_ENABLE_CUDA=ON
```

cmake/CudaTargets.cmake adds CUDA libraries, executables, and GPU tests. The separate cuda/CMakeLists.txt entry point also enables the CUDA graph.

CUDA builds require a compatible NVIDIA GPU, driver, toolkit, and C++ host compiler. The recorded validation environment used CUDA Toolkit 13.4 and an RTX 4060 Laptop GPU with compute capability 89. That environment record is evidence of one tested setup, not a guarantee for every machine.

5. What validation means here

The cleanup record reports that the final CUDA regression suite passed 8/8. Independent Decimal80-oriented CUDA checks reported zero failures across recorded SPAI, GMRES, and iterative-refinement snapshots. CLI checks covered both normal and error paths.

This demonstrates bounded numerical correctness on the supplied deterministic tests. It does not by itself establish:

- a benchmark speedup;
- universal convergence for every sparse matrix;
- bitwise CPU/GPU equality;
- a mixed-precision convergence guarantee;
- a novel GPU SPAI algorithm.

6. Known limitations to disclose when relevant

- GPU and CPU native patterns may differ on exact candidate-score ties.
- Singular QR failure statuses may differ between CPU and GPU.
- GPU FP32 score-history/status diagnostics are more limited than the FP64

CPU diagnostic records.

- FP32 duplicate-entry summation keeps the original order and is not presented

as a redesigned reproducible reduction scheme.

- Unrestarted GMRES can require substantial memory for large systems.
- The active solver does not implement mixed precision or BSPAI bucketing.

Suggested speaking script (about 70 seconds)

"We maintain separate FP64 and FP32 implementations because float precision has much less precision and a smaller numerical range. The FP32 path therefore uses its own matrix type, tolerances, diagnostics, and range checks rather than blindly replacing double with float. The CUDA backend maps the main sparse operations, SPAI construction, and GMRES work onto the GPU, while the outer refinement coordination remains CPU-staged. We validate numerical behavior against deterministic tests and independent checks. Our results support correctness for those tests; they do not claim a general performance speedup or a completed mixed-precision/BSPAI implementation."

Likely questions and answers

7. The four implementations and their real entry routes

Variant - SPAI construction - GMRES correction - Outer refinement and final check

CPU FP64 - CPU FP64 - CPU FP64 - CPU FP64

CPU FP32 - CPU FP32 - CPU FP32 - CPU FP32

CUDA FP64 - Native GPU SPAI - GPU GMRES - CPU-staged control/check

CUDA FP32 - Native GPU FP32 SPAI - GPU FP32 GMRES - CPU-staged control/check

The main native FP64 route is cuda_gpu_spai_gmres_ir in cuda_ir.cpp. It calls cuda_construct_spai in cuda_spai.cu, then passes the resulting preconditioner to cuda_refine_with_preconditioner. The latter owns a CudaGmresContext from cuda_gmres.cu.

The FP32 counterpart is fp32_cuda_spai_gmres_ir, using fp32_cuda_spai.cu, fp32_cuda_ir.cpp, and fp32_cuda_gmres.cu.

There is also a hybrid diagnostic function called cuda_spai_gmres_ir. Its similar name can be misleading: it can use CPU SPAI with CUDA GMRES. Use the cuda_gpu_... name when discussing native GPU SPAI construction.

8. CUDA vocabulary from zero

- Host: the CPU and its normal RAM.
- Device: the GPU and its separate device memory.
- Kernel: a function executed in parallel by many GPU threads.
- Thread: one small GPU worker executing a kernel instance.
- Block: a cooperating group of GPU threads.
- Stream: an ordered queue of GPU operations; independent streams can overlap work where dependencies permit.
- cuSPARSE: NVIDIA library used for sparse operations such as CSR SpMV.
- cuBLAS: NVIDIA library for vector and dense linear-algebra operations such as dot products and axpy (y = alpha*x+y).
- cuSOLVER: NVIDIA library used here for QR-related dense solves.
- Descriptor: a library object describing a matrix or vector's shape, datatype, and memory location.
- Workspace: temporary memory a library routine needs while performing an operation.

Moving data across the CPU/GPU boundary costs time. A good GPU implementation therefore considers both arithmetic parallelism and the lifetime of device data.

9. GPU SPAI internals

cuda_construct_spai uploads A and performs the following stages:

- 1. A GPU kernel scales the matrix and prepares the orientation used by the construction.
- 2. The host owns the irregular pattern and shadow-set bookkeeping. These sets vary by column and step, so they are less regular than bulk arithmetic.
- 3. Work is grouped into a small batch of slots, with up to eight CUDA streams used to overlap independent column work.
- 4. GPU code gathers each reduced dense matrix needed by a column's least-squares problem.
- 5. cuSOLVER performs QR factorization and applies the orthogonal transformation.
- 6. A QR guard checks whether the small triangular system is safe to solve.
- 7. cuBLAS solves the triangular system for the fitted column coefficients.
- 8. A fused GPU path evaluates the fitted residual and reuses it for candidate scoring and diagnostics.
- 9. The host selects pattern-growth candidates according to the algorithm's rules.
- 10. A GPU assembly stage produces the sparse preconditioner, which is canonicalized and made available to the refinement route.

An experimental row-parallel candidate-scoring design was measured and rejected because it was 5-7% slower on the studied workload. This is a useful research result: an optimization idea is not retained merely because it sounds more parallel.

10. GPU GMRES internals

The context constructor uploads A and P, creates a stream, cuBLAS/cuSPARSE handles and descriptors, reusable work vectors, and library workspaces. One context can then solve more than one outer correction without recreating these resources.

During one GMRES solve:

- 1. The right-hand side and starting data are uploaded.
- 2. cuSPARSE applies A and P; the product P*A is never explicitly formed as a new sparse matrix.
- 3. A safe scaled-norm kernel computes vector norms without needless overflow or underflow risk.
- 4. cuBLAS dot products and vector updates implement Arnoldi orthogonalization.
- 5. Full basis vectors remain on the GPU.
- 6. The small Hessenberg matrix, Givens rotations, and triangular coefficients are managed on the CPU. Small scalar values therefore cross the boundary during iterations.
- 7. The GPU constructs d=d0+V*y.
- 8. The actual operator is applied again to verify ||P*r-P*A*d||_2 / ||P*r||_2.
- 9. The final correction and requested history are downloaded.

The matrix is not copied back and forth at every iteration. In native GPU SPAI, A is uploaded for construction; the resulting P is made available on the host; then A and P are uploaded into the GMRES context and retained for refinement corrections. This is not yet a fully device-resident end-to-end pipeline, because the outer IR control remains CPU-staged.

11. FP32 is a numerical design, not a spelling change

FP32 has about 24 binary digits of significand precision, while FP64 has about 53. Rounding appears sooner in dot products, norms, QR solves, orthogonalization, residual formation, and duplicate summation. Therefore the FP32 implementation needs:

- FP32 storage and arithmetic where the variant promises FP32;
- single-precision cuBLAS/cuSOLVER routines, for example S rather than D variants;
- FP32-appropriate constants and thresholds;
- explicit checks for NaN and infinity;
- validation against independent high precision;
- tests that catch accidental promotion to double or precision-policy drift.

The shared CUDA SPAI body is specialized through macros for FP32: scalar type, cuSOLVER calls, cuBLAS triangular solve, and result mappings change together. That reduces duplicated control logic while keeping precision-specific semantics visible.

The correction verification threshold is 1e-4 for FP32 versus 1e-8 for FP64. The whole-solution bound is based on roughly 10*2^-24 versus 10*2^-53. These are deliberate precision-aware policies, not evidence that FP32 is "wrong."

12. Persistent context and lazy basis storage

Originally, repeated FP32 correction solves could recreate GPU resources and upload invariant matrices. Fp32CudaGmresContext now retains one device copy of A and P, the stream, handles, descriptors, eight reusable device work vectors, and workspaces. The outer code constructs it lazily when the first correction is actually required and reuses it afterward. If the initial solution already satisfies the outer check, no context is needed.

Unrestarted GMRES can in principle need many basis vectors. Eagerly reserving all possible vectors wastes

both memory and initialization time on easy problems. The FP32 context now grows device basis memory in stable slabs of 16 vectors and the host projected system in 16-column chunks.

For n=12000, the former eager device-basis request was 576,048,000 bytes. A one-step solve now starts with one 768,000-byte slab and 1,088 bytes of host Hessenberg storage. An exact initial solution requests neither. "Lazy" therefore means allocate only when a calculation proves the storage is needed.

13. Results and the correct interpretation

- The active CUDA suite passed 8/8 CTest tests, and all four solver command-line programs passed 32 Decimal80 checks.
- Persistent-context isolated fixed-P refinement medians improved from 10.50-10.85 ms to 9.71-9.80 ms, about 7-10%. Complete GPU SPAI-GMRES-IR stayed around 2.15-2.16 s, so construction dominated enough that the end-to-end change was unclear.
- For a size-12,000 identity case, lazy storage reduced a one-step run from 220.103 ms to 1.331 ms and an exact-initial case from 16.634 ms to 0.907 ms.
- On one stencil_8192 FP32 workload, pooled medians changed from 2639.473 ms to 2290.805 ms, 13.21% lower or about 1.15x. Status, construction counts, two outer steps, fifteen inner iterations, backward error, and serialized FP32 solution matched.
- The fused SPAI residual path produced an exploratory constructor median near 2153 ms versus 2282-2307 ms, approximately 5-7% lower.

These measurements show particular benefits on one GPU and specific matrices. They do not prove the same percentage for every matrix or device. Always attach workload, precision, and validation statement to a timing claim.

14. What remains imperfect

CPU and GPU SPAI may choose different candidates when scores tie, while both remain valid. Singular QR cases can produce different low-level statuses. GPU FP32 score-history/status mapping is not as rich as the CPU path. Duplicate Matrix Market entries retain the original FP32 summation ordering. Lazy slabs remove waste for short solves, but a truly long unrestarted solve must still store a growing basis. The outer GPU refinement is CPU-staged, so "fully GPU-resident solver" would be inaccurate.

15. CUDA files, from executable to kernel

- cmake/CudaTargets.cmake: creates CUDA SpMV, GMRES, SPAI, IR, tests, and four solver executables; links CUDA Runtime, cuSPARSE, cuBLAS, and cuSOLVER.
- include/bspai/cuda_spmv.hpp, src/cuda_spmv.cu: reusable FP64 CSR SpMV object and cuSPARSE resource ownership.
- include/bspai/fp32_cuda_spmv.hpp, src/fp32_cuda_spmv.cu: FP32 counterpart.
- include/bspai/cuda_spai.hpp, src/cuda_spai.cu: FP64 GPU SPAI public result/telemetry and implementation.
- include/bspai/fp32_cuda_spai.hpp, src/fp32_cuda_spai.cu: FP32 interface and precision specialization of the shared SPAI body.
- include/bspai/cuda_gmres.hpp, src/cuda_gmres.cu: FP64 one-shot and persistent-context GMRES.
- include/bspai/fp32_cuda_gmres.hpp, src/fp32_cuda_gmres.cu: FP32 context, lazy basis slabs, one-shot wrapper, and CPU-staged FP32 refinement.
- include/bspai/cuda_ir.hpp, src/cuda_ir.cpp: FP64 CPU-staged refinement and complete CPU-SPAI/native-GPU-SPAI routes.
- src/solver_main.cpp: common CLI and output.
- CUDA tests: test_cuda_spmv.cpp, test_cuda_gmres.cpp, test_cuda_ir.cpp, test_cuda_spai.cpp, test_cuda_spai_sparse.cpp, plus FP32 counterparts and cuda_spai_allocation_audit.cu.

16. Legend: exactly what is parallel and serial

The following labels are used below.

- [HOST SERIAL] one CPU thread executes control flow, set manipulation, small solves, status mapping, or output.
- [GPU KERNEL PARALLEL] a custom CUDA kernel launches many GPU threads over rows, entries, or candidates.
- [GPU LIBRARY PARALLEL] cuSPARSE, cuBLAS, or cuSOLVER launches its internal parallel GPU work.
- [STREAM-CONCURRENT] independent column slots are queued on separate nonblocking CUDA streams and may overlap.
- [SYNCHRONIZATION] the CPU must wait because it needs a result, a buffer is being reused, or lifetime safety requires completion.
- [TRANSFER] bytes cross between host RAM and GPU memory.

CUDA kernel launches are asynchronous with respect to the CPU unless synchronization or a dependent operation forces waiting. Operations submitted to one stream execute in order. Separate streams may overlap when hardware resources and dependencies permit; this is potential concurrency, not a promise that every launch overlaps perfectly.

17. Native CUDA SPAI: complete top-to-bottom flow

Public function:

```
CudaSpaiResult cuda_construct_spai(
    const CsrMatrix& a,
    const SpaiOptions& options,
    int device = 0);
```

Input a must be square, canonical CSR and finite. options supplies epsilon, alpha, and beta. device is the CUDA device index. Output contains ordinary SpaiResult construction plus CudaSpaiTelemetry telemetry.

- 1. [HOST SERIAL] Validate dimensions/options; create Guard to select and later restore the CUDA device.
- 2. [HOST SERIAL] Construct RAII Memory, Stream, Handles, and up to eight Slot objects. RAII means destructors release CUDA resources even if an exception occurs.
- 3. [TRANSFER H2D] Upload CSR row offsets, column indices, and values with cudaMemcpyAsync.
- 4. [GPU KERNEL PARALLEL] scale_matrix(offsets,columns,values,n,d,b,status) assigns work across matrix rows/nonzeros, computes dyadic scaling d, and produces sparse scaled orientation B values.
- 5. [TRANSFER D2H + SYNCHRONIZATION] Download scale status, scaling, and sparse B values because the adaptive host controller needs canonical structural metadata.
- 6. [HOST SERIAL] Initialize one pattern per output column and a done flag. Iterate growth rounds.
- 7. [HOST SERIAL] For each active column, calculate shadow rows/candidates, filter existing pattern positions, sort metadata, and size its reduced problem.
- 8. [STREAM-CONCURRENT] Assign independent columns to available slots/streams. Each slot maintains reusable buffers rather than allocating every fit.
- 9. [TRANSFER H2D] Enqueue current shadow, pattern, and target data for the slot.
- 10. [GPU KERNEL PARALLEL] gather uses GPU threads over reduced rows/elements to materialize the column-major dense reduced matrix needed by QR.
- 11. [GPU LIBRARY PARALLEL] cusolverDnDgeqrf performs FP64 QR factorization. cusolverDnDormqr applies Q^T to the target vector. FP32 specialization calls the corresponding Sgeqrf/Sormqr routines.
- 12. [GPU KERNEL PARALLEL] qr_guard checks pivots/rank safety before the triangular solve.
- 13. [GPU LIBRARY PARALLEL] cublasDtrsv or cublasStrsv solves the upper-triangular R*m=y system.
- 14. [GPU KERNEL PARALLEL] fitted_residual evaluates reduced/full residual information. The retained optimization reuses this fitted residual instead of recomputing the same work for scoring and diagnostics.
- 15. [GPU KERNEL PARALLEL] scores evaluates candidate improvement data across candidates;

single_score supports the standalone scoring API.

- 16. [TRANSFER D2H + SYNCHRONIZATION] Collect coefficient, residual, status, and score records only when the host adaptive decision needs them.

- 17. [HOST SERIAL] Sort candidate indices deterministically, apply the score/mean policy, choose at most beta, grow pattern, update histories/status, and decide which columns continue.

- 18. Repeat steps 7-17 for at most the algorithmic growth allowance.

- 19. [HOST SERIAL] Pack solved column row indices and coefficients into sparse output arrays.

- 20. [TRANSFER H2D] Upload packed rows, coefficients, and scaling.

- 21. [GPU KERNEL PARALLEL] assemble_p(coefficients,rows,d,...) scales and checks sparse preconditioner values in parallel.

- 22. [TRANSFER D2H + SYNCHRONIZATION] Download packed P and assembly status; create canonical host CSR matrices and return the numerical result plus telemetry.

The outer adaptive rounds are serial because later patterns depend on earlier fits. Parallelism exists inside every dense/sparse numerical fit and across independent columns submitted to separate streams.

18. CUDA GMRES: complete top-to-bottom flow

FP64 context constructor:

```
CudaGmresContext(A, P, options, device);
CudaGmresResult solve(rhs, initial_guess = {});
```

FP32 context differs slightly: options are passed to each solve, allowing the same immutable matrices to serve sequential solves with different GMRES settings.

Context construction and persistent buffers

[HOST SERIAL] Validate canonical square A and P, select device, create a nonblocking stream, cuBLAS handle, cuSPARSE handle, sparse descriptors, and dense-vector descriptors. Math modes are checked; FP64 requests pedantic/native arithmetic and disables atomic reductions where required for policy.

[TRANSFER H2D] Upload for each matrix: 64-bit CSR row offsets, 64-bit column indices, and FP64/FP32 values. matrix_uploads() should report two immutable matrix initializations: A and P.

The context owns reusable full-length device vectors for RHS, iterate, initial iterate, residual/intermediate vectors, candidate/update work, and flags/norm outputs. Each sparse matrix also owns a cuSPARSE workspace sized by cusparseSpMV_bufferSize. FP32 Krylov basis storage is separate and grows in slabs of 16 vectors; host Hessenberg capacity grows in 16-column chunks.

One solve

- 1. [HOST SERIAL] Validate RHS/initial dimensions and option bounds. Handle zero-length/invalid cases.

- 2. [TRANSFER H2D] Upload RHS, current initial guess, and preserved initial guess.

- 3. [GPU LIBRARY PARALLEL] cuSPARSE applies A*x; cuBLAS subtraction forms r=rhs-A*x; cuSPARSE applies P*r.

- 4. [GPU KERNEL/LIBRARY PARALLEL] compute safe norms, finite checks, and normalization. Easy exits occur before Krylov basis allocation where possible.

- 5. [GPU LIBRARY PARALLEL] Copy the normalized residual to basis vector v0.

- 6. For Krylov iteration k:

- [GPU LIBRARY PARALLEL] cuSPARSE computes A*v_k, then P*(A*v_k).

- [GPU LIBRARY PARALLEL] cuBLAS dot products compute Arnoldi coefficients against prior basis vectors.

- [SYNCHRONIZATION / scalar D2H] each host-observed dot/norm scalar is needed to update the serial small Hessenberg problem.

- [GPU LIBRARY PARALLEL] cuBLAS AXPY removes each projection; the safe norm obtains the subdiagonal; a division kernel normalizes the new basis vector.

- [HOST SERIAL] update Hessenberg entries, apply earlier Givens rotations, compute the new Givens rotation, and update g and the estimated residual.
- When verification is triggered, [HOST SERIAL] solve the small triangular system for y; [GPU LIBRARY PARALLEL] form d=d0+V*y; then apply actual A and P and recompute the true correction ratio.
- 7. [TRANSFER D2H + SYNCHRONIZATION] Download final correction and retained histories after termination.
- 8. Return numerical GmresResult plus transfer counters in FP64.

The Arnoldi loop has a serial dependency between iterations: iteration k+1 needs v_{k+1} from iteration k. Within each iteration, vector-wide SpMV, dot, AXPY, scaling, validation, and norm operations are GPU-parallel. The small Hessenberg/Givens work remains serial because its size is iteration-count scale rather than problem-size scale.

19. CUDA Runtime, cuSPARSE, cuBLAS, and cuSOLVER calls used

CUDA Runtime

- cudaGetDevice / cudaSetDevice: query/select GPU.
- cudaMalloc / cudaFree: allocate/free device buffers.
- cudaMemcpyAsync: enqueue H2D or D2H copies on a stream.
- cudaMemsetAsync: initialize output buffers/flags.
- cudaStreamCreateWithFlags(...NonBlocking): create command queues.
- cudaStreamSynchronize: establish completion before host use or buffer reuse.
- cudaGetLastError: catch kernel launch errors.

cuSPARSE

- cusparseCreate and cusparseSetStream: create a sparse-library context attached to a stream.
- cusparseCreateCsr: describe CSR topology and values without copying them.
- cusparseCreateDnVec: describe dense input/output vectors.
- cusparseDnVecSetValues: repoint a reusable vector descriptor at another device buffer.
- cusparseSpMV_bufferSize: ask how much temporary memory an SpMV algorithm needs.
- cusparseSpMV: perform parallel sparse matrix-vector multiplication.
- destroy calls release descriptors/handles.

cuBLAS

- cublasDcopy_64 / cublasScopy_64: device vector copy.
- cublasDaxpy_64 / cublasSaxpy_64: parallel y=alpha*x+y.
- cublasSscal_64: FP32 vector scaling; FP64 normalization also has a custom division path.
- cublasDdot_64 / cublasSdot_64: reduction for inner products.
- cublasSnrm2_64: FP32 Euclidean norm where used; FP64 has a custom safe-scaled norm kernel.
- cublasDtrsv / cublasStrsv: triangular solve for SPAI's QR result.
- stream, pointer-mode, math-mode, and atomics-mode calls configure numerical/execution policy.

cuSOLVER

- cusolverDnCreate / cusolverDnSetStream: solver handle per SPAI slot.
- deterministic/math-mode getters and setters enforce requested behavior.
- cusolverDnDgeqrf_bufferSize and Dormqr_bufferSize: query reusable workspace.
- cusolverDnDgeqrf / Sgeqrf: QR factorization.
- cusolverDnDormqr / Sormqr: apply Q^T to the target.
- devInfo values are downloaded and checked before treating the fit as valid.

20. Buffer ownership and transfer details

The Memory/Mem wrappers own raw device pointers and free them in destructors. Matrix owns the CSR device arrays, descriptor, and SpMV workspace. Engine owns stream/library handles and common vector descriptors. Run owns per-solve vectors and Krylov storage. Slot owns one CUDA SPAI stream, cuSOLVER/cuBLAS handles, reduced dense matrix, QR factors/workspace, shadow/pattern/candidate arrays, residuals, coefficients, score wires, and status buffers.

CUDA SPAI telemetry counts source-issued allocation/free/reallocation calls, cumulative/peak/current bytes, largest allocation, sparse B bytes, packed-output bytes, reduced dimensions, QR workspace, nonzeros, transfer counts/bytes/size buckets, synchronization counts, streams, batch width, QR submissions, and GPU score evaluations. Vendor-internal allocations are explicitly excluded.

GMRES transfer telemetry separates vector uploads/downloads, Arnoldi-vector movement, scalar observations, dot/norm/finite-flag observations, and host scalar inputs to AXPY, SpMV, and division. This helps distinguish unavoidable algorithmic dependencies from accidental full-vector traffic.

21. CUDA optimization history and evidence

- 1. Sparse rather than dense GPU SPAI storage. B remains CSR, reduced dense matrices are sized to actual shadow-by-pattern problems, and output is packed by solved nonzeros. Allocation-audit tests confirm topology/value sizes and release of source-owned allocations.
- 2. Multiple reusable fit slots/streams. Up to eight independent SPAI column fits can be submitted before collection. This exposes column-level concurrency and reuses slot capacity.
- 3. Fused fitted residual reuse. Residual work feeds scoring and diagnostics. Recorded exploratory constructor median: about 2153 ms versus 2282-2307 ms, around 5-7% lower.
- 4. Rejected row-parallel scoring experiment. It was about 5-7% slower on the tested workload, so it was not retained.
- 5. Persistent FP32 GMRES context. A, P, handles, descriptors, eight work vectors, and workspaces survive across outer corrections. Isolated fixed-P medians improved from 10.50-10.85 ms to 9.71-9.80 ms, about 7-10%; whole-solver time stayed around 2.15-2.16 s.
- 6. Lazy FP32 Krylov slabs. Basis grows in 16-vector slabs. At n=12000, the old 576,048,000-byte eager request became one 768,000-byte slab plus 1,088 host bytes for a one-step case; exact initial solutions allocate neither. Runtime changed 220.103 to 1.331 ms for the one-step identity and 16.634 to 0.907 ms for exact initial. On recorded stencil_8192, 2639.473 to 2290.805 ms was 13.21% lower/about 1.15x with matching outputs and iterations.

All timings are diagnostic host wall-clock observations on specified workloads and one GPU. They do not prove universal scaling, and the SPAI telemetry fields explicitly say they are not CUDA event timings used in numerical decisions.

Why is FP32 useful? It can reduce storage and can be faster on suitable hardware, but it has lower accuracy and must be validated independently.

Why are FP32 tolerances looser? Roundoff in float is much larger, so an FP64-level tolerance is usually not attainable or meaningful in FP32.

Does GPU execution guarantee the same output as CPU? Not bit-for-bit. Different operation orders can change rounding; we assess numerical agreement through residual and backward-error checks.

Is this mixed precision? No. Each active solver uses one arithmetic precision throughout its main numerical path.

Is the whole solve on the GPU? The principal CUDA kernels are on the GPU, but the outer iterative-refinement loop is CPU-staged in this project.